

基于 Unity3D 的动作角色扮演游戏战斗系统研究与开发

何柳青

(广州工商学院, 广东 广州 510850)

摘要: 游戏开发已成为数字媒体技术专业的重要课程内容之一, 其中动作角色扮演类游戏在游戏市场中所占比重较大。以 Unity3D 游戏引擎开发第三人称动作角色扮演游戏为例, 结合 3ds Max 和 Visual Studio 进行游戏开发。使用 C# 进行场景编程实现角色行为(包括行走、攻击、怪物 AI 等)控制, 调整适合玩家的战斗模式, 并对游戏战斗模块进行封装优化, 有效降低游戏运行的功耗, 增加整体流畅度, 提升游戏玩家体验。

关键词: 动作角色扮演; 角色行为控制; 战斗模式; 封装优化

中图分类号: TP317

文献标识码: A

文章编号: 2096-4706 (2023) 24-0001-05

Research and Development of Action Role-Playing Game Battle System Based on Unity3D

HE Liuqing

(Guangzhou College of Technology and Business, Guangzhou 510850, China)

Abstract: Game development has become one of the important course contents for digital media technology majors, in which, action role-playing games accounting for a large proportion in the game market. This paper takes Unity3D game engine development of third person action role-playing games as an example, and combines 3ds Max and Visual Studio for game development. By using C # to program scenes to control character behavior, including behaviors such as walking, attacking, and monster AI, adjusting the combat mode suitable for players, and encapsulating and optimizing game combat modules, the power consumption of game operation is effectively reduced, the overall smoothness is increased, and the game player experience is enhanced.

Keywords: action role-playing; role behavior control; combat mode; packaging optimization

0 引言

随着硬件性能的提高, Unity3D 游戏引擎以 GPU 实时渲染生成高帧画面, 给玩家带来高品质的游戏体验, 同时玩家也对战斗体验有了更高要求, 设计优质的战斗系统已成为衡量角色扮演游戏优劣的重要保证。而战斗系统的设计与制作涉及多个重要环节, 各个环节之间紧密相连。

1 游戏战斗系统框架设计

本项目的战斗系统框架分为战斗方式、战斗攻防模型和战斗元素三个部分。

1.1 战斗方式

战斗方式是战斗系统中最直观的环节, 其设计具有多样化, 可以是多种设计合理地整合在一起, 以更好地呈现出战斗的紧张感和趣味性。战斗方式并非固定不变, 而是根据游戏地图环境的变化而有所变动。

动作角色扮演游戏的战斗方式大致分为以下 4 点:

- 1) 玩家需要在怪物的攻击群中生存, 或者是击杀一定数量的普通怪物和击杀少数的首脑怪物。
- 2) 通过孵化器孵化出一定数量的怪物, 并通过体积碰撞或投掷物来攻击玩家。
- 3) 刷新不同批次的怪物, 后续批次怪物的战斗能力比前面的怪物稍强。
- 4) 玩家可以通过武器和技能来击杀怪物, 同时以移动、闪避的方式躲避怪物的攻击。

1.2 战斗攻防模型

游戏中的我方或敌对角色在战斗系统中只有四种状态: 移动、攻击、防御、闪避。当主角遇到怪物时, 通过空间位置移动让怪物进入攻击范围。在这个过程中, 双方均没有发起攻击行为, 处于备战状态, 可以实施移动和防御两种行为。假设主角实施攻击行为, 则双方进入攻防状态^[1]。当主角对受伤方发起攻击时, 怪物有三种可选行为: 不防御且同时进攻、防御、闪避。攻击方发起且完成一次攻击后, 即完成了一次攻防流程, 双方再度进入备战状态。处于备战状态的双方计划下一步的策略是进攻、防御还是闪避, 如其中一方受到严重伤害——死亡则结束战斗。攻防模型

收稿日期: 2023-05-16

基金项目: 2022 年广州工商学院校级
科研项目“基于 Unity3D 的 ARPG 游戏战
斗系统研究与实现”(KYZC202211)

如图 1 所示。

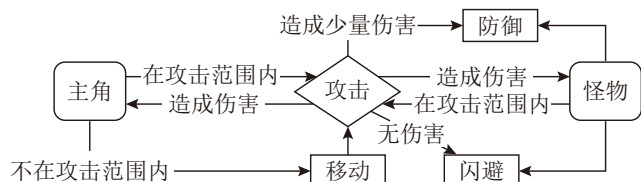


图 1 攻防模型

主角是主动发起攻击的一方，所以在攻防模型中不存在主角防御或闪避的行为。而在攻防过程中，移动是没有直接收益的，但它可以令攻击行为获得最大收益，如移动到某个角度打击要害，或移动中实施攻击。所以说移动是战斗过程中较具灵活性的行为，主要用来辅助攻击和对抗攻击。在图 1 的攻防模型中，闪避在游戏设计中也视为防御，无论是攻击方主动发起攻击行为，还是受击方发起防御或闪避行为均形成攻防关系。

1.3 战斗元素

战斗元素与攻击招式相关，普通攻击和技能攻击中均有涉及。

1.3.1 攻击范围

包含普通攻击范围和技能攻击范围，因技能需要消耗角色自身部分能量，故技能攻击范围必定大于等于普通攻击范围，这样才能确保收益平衡。

1.3.2 招式前摇

招式包括普通攻击和技能攻击的动作，无论何种攻击，在受击对象被判定受到攻击前的时间均属于前摇耗时，即产生攻击收益前的时间。这段时间是无法产生收益的，但却是获得攻击收益必不可少的步骤。前摇时间越短，越早对敌方发起攻击，反之越晚对敌方发起攻击。通常情况下，攻击力强的招式前摇时间较长，反之则前摇时间较短。因此，玩家在即时战斗前就需要考虑如何合理组合攻击招式以获得最大收益。

1.3.3 攻击硬直

招式在击中敌人后，敌人会有短暂的动作停滞而无法活动，通常不会超过一秒。在攻击硬直时间内，受击者无法做出任何行动，容易被攻击者施放连招而造成更大的伤害。

1.3.4 招式后摇

玩家角色在施放完攻击动作后，如没有继续输入攻击指令则会产生收招动作，而收招动作属于后摇。后摇所需的时间会直接影响整体攻击的频率。如果后摇时间较短，则下一次攻击会更早，从而提高攻击频率。

从图 2 中可以看出，在攻击范围内，在敌人无闪避行为的前提下击中敌人会令敌人产生攻击硬直状态，但同时也会令自身角色产生招式后摇，这两者是同时发生的，只是两种状态依附的角色对象不同而已。



图 2 战斗元素关系

2 战斗的多重机制

2.1 战斗的对抗机制

在常规的战斗环境中，怪物不会比主角强太多，玩家只需采取一些策略便可以通过关卡。一般情况下，平衡的对抗关系是使玩家持续玩下去的动力之一，也是战斗对抗机制的一个要素。然而平衡的对抗关系不能给游戏带来更多的刺激性和挑战性，因此在某些环境下打破这个平衡关系能给游戏增添更多的可玩性。游戏中的不同角色、武器、技能具有不同的作用，角色与角色之间有所差异，角色与怪物之间更是迥然不同。对付同一种怪物，不同的角色有不同的战斗机制。这样可避免单一角色的同质化，使得每一个角色都能专精本职业的玩法策略，获得差异化的战斗体验^[2]。然而，实现这种对抗机制的前提条件是保证角色的战斗能力综合评价不能远低于敌人，避免任何一方以绝对强势的战力破坏对抗机制。

战斗可以增加某一方的人数形成差异，如增加敌人数量，形成双方人数不平衡的对抗关系。但双方人数上的较大差异并非能够形成绝对优势，通常只是弥补单体敌人战力的不足，从而更好地维持战力平衡，这种机制通常用在对抗普通怪物的环节中。

在对抗 BOSS 强敌的过程中，不能以增加数量的方法来加强怪物战力，BOSS 强敌本身的战力就比玩家角色强，但玩家不能直接依靠自身的战力来战胜它，因此需要设置特定的战斗机制来解决这种问题。例如使用特定的技能、触发场景的机关、特定时间段行动等可以大幅度提高伤害数值或削弱强敌的防御力，从而在伤害输出方面带来更高的收益，更容易击败强敌，以弱胜强。玩家在通过机制战胜强敌后会获得更大的成就感。对抗机制关系如图 3 所示。

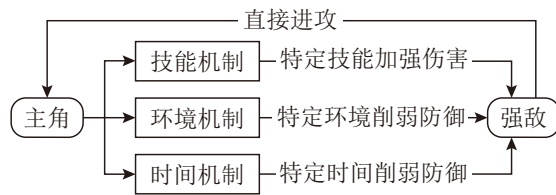


图 3 对抗机制

2.2 战斗的交互机制

角色在发起攻击或防御时会播放相应的动作动画，而双方在动作发生碰撞或进入攻击范围时便产生了交互。攻击或防御动作具有空间判定性，攻击判定

和受击判定都是针对不同的部位。如武器击中对方身体产生了交互，会播放受击者被击中后的硬直动画，由于硬直时间非常短，无法做出攻击、防御、闪避的行为，这时候攻击者更容易发起连续攻击。

交互机制也遵循对抗机制，但它却有不确定性，决策过程完全取决于玩家选择何种行为。玩家采取何种策略，会带来不同的对抗过程和对抗结果，这一点

无法通过游戏规则来限制其发展。例如玩家在遇到敌人时可以利用地形优势发起远程攻击，或直接远离敌人的攻击范围令其暂时无法攻击，等待夜晚怪物入睡后再进行突袭。交互机制存在多种可能性，玩家可以

将技能、武器、道具、环境等因素纳入对抗策略中，而交互机制的设计思想是只提供对抗条件和任务目标但不限制策略，如图 4 所示。

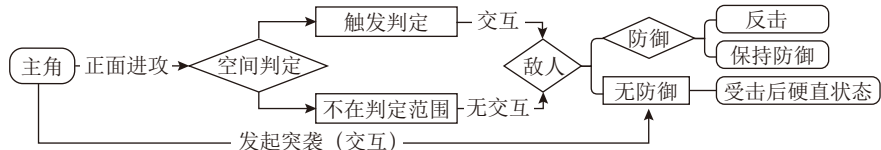


图 4 交互机制

2.3 战斗关卡的运行机制

一个战斗关卡可以存在一个或多个场景，而关卡运行机制则融合了至少一个场景的相关元素。关卡主要元素包括时间、空间、即时道具，这三个方面均是关卡自身产生的元素，同时它们也会直接影响战斗机制和战斗结果，是游戏关卡设计的必要因素。主角和怪物都可以在关卡机制中得到相应的强化或弱化，如何运用关卡机制来取胜是玩家必须要思考的问题，这就进一步增强了玩家在游戏

2.3.1 关卡时间

中的互动性。游戏战斗关卡的通关条件之一是战胜怪物敌人的时间不超过关卡的规定时限，反之则判定为通关失败。不同场景（地图）的关卡时限不同，与普通怪物和 BOSS 怪物战斗的时限也不同。在规定的时限内，需要基于关卡时间来计算玩家时间的消耗，以方便玩家即时了解时间消耗而实施相应的战斗策略。

2.3.2 关卡空间

对于关卡空间，在玩家实施战斗的过程中可以设定为有空间限制或无空间限制。通常在通关式的游戏中均有空间限制，在有限的空间内与敌人战斗直至其中一方获胜。关卡地图设定了边界，玩家需要结合活动范围来制定战斗策略。有限的空间会影响玩家武器的使用和技能的施放，提高了生存的难度。

2.3.3 关卡道具

关卡道具特指关卡里掉落

的物品，只对关卡本身起到提升作用，而它需要玩家角色拾取才能产生效果，如玩家没有拾取到相应道具则不会产生效果，且在限

3 角色行为模块

定时间内玩家未拾取到相应的关卡道具，其可能会消失。关卡道具可以在短时间内令玩家角色的战力得到一定幅度的提升，从而影响游戏难度曲线，降低战斗难度且可选择采用更多可行性的战斗策略。部分角色扮演游戏都会存在战斗系统，这部分游戏被列为动作角色扮演游戏（Action Role Playing Game, ARPG）。而在 ARPG 游戏中，无论是玩家还是“怪物”，均可以设计一套战斗行为模块。这套行为模块主要用来设定战斗标准，该模块能够减少游戏冗余数据，降低计算量且提高游戏的运行速度，这是游戏优化的条件之一。

3.1 通用行为模块

功能组模主要是为了实现移动、动画、攻击、寻路等。

- 1) 移动模块。控制角色的移动。
- 2) 动画模块。控制角色不同状态动画的无缝切换。
- 3) 攻击模块。计算攻击产生的伤害与即时状态，但不计算伤害的具体数值。
- 4) 寻路模块。控制角色自动寻路或手动寻路。

这部分组模代码在游戏里基本通用，只提供机制但不提供具体计算方法和数值。

3.2 AI 行为模块

游戏中存在多种行为，玩家可以手动操作自己的角色，而怪物则需要借助 AI 行为去驱动。战斗系统中的行为比较复杂，因此需要将行为分成多个模块进行管理，如图 5 所示。

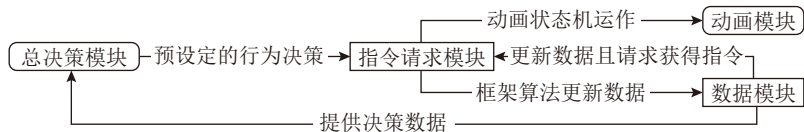


图 5 AI 行为模块结构关系

1) 总决策模块。怪物在原地待机时, 假设玩家进入其攻击范围。怪物以预设定的行为对玩家发起攻击。

2) 指令请求模块。判定请求的指令是否超出预置规则范围, 如怪物是否有负面状态导致行为受限。如果怪物的状态正常则向下层模块发出攻击行为请求。

3) 动画模块。怪物在移动或攻击时播放相应动画。

4) 数据模块。用于怪物的各种状态数据(包括怪物的坐标数据和自身状态数据)更新。数据更新的同时请求相应指令, 同时向总决策模块提供信息, 决策模块检查数据并发出同步决策指令。

3.3 区域 AI 模块

AI 可以根据不同情况进行划分, 具体包括区域 AI、队列 AI 和单体 AI。

一张地图上存在多个区域, 不同区域可以单独设置 AI。区域 AI 容易与关卡运行机制相混淆。关卡运行机制主要是针对如何击败怪物而设, 而区域 AI 则直接作用于怪物, 令怪物具有模拟人类思考的行为, 因此这两者所针对的主体有本质上的区别。

队列 AI 主要控制整队怪物的行为策略, 单体 AI 则控制单个怪物的行为策略。例如, 玩家进入到地图的某一区域, 该区域有一队共 5 个怪物, 区域 AI 则会检测哪一个怪物离玩家最近, 单体 AI 此时发出攻击玩家的指令, 同时由队列 AI 发起信号让全队怪物进攻玩家。而其他 4 个怪物具体如何进攻则可以由队列 AI 和单体 AI 同时控制, 比如队列 AI 会发出围攻玩家的指令, 攻击则由单体 AI 控制。由此说明这三种 AI 并不冲突, 是可以并存的。

设定怪物有待机、走路^[3]和死亡三种状态, 现针对待机 and 走路两种状态进行 AI 控制, 实现单体怪物 AI 部分行为的 C# 代码如下:

```
public enum MonsterCondition
{
    Idle, Walk, Attack, Death
}

public string monster_present; // 怪物当前状态
public string monster_idle; // 怪物待机
public string monster_walk; // 怪物走路
void RandomState()
{
    int value = Random.Range(0, 2); // 定义一个随机整数, 取 0 和 1
    if (value == 0)
    {
        monster_present = monster_idle; // 当随机值为 0 时播放怪物 Idle 动画
    }
    else
```

```
{ // 当随机值为 1 时
    if (monster_present != monster_walk)
    {
        transform.Rotate(transform.up * Random.
Range(0, 360));
        // 怪物沿竖直方向旋转随机角度
    }
    monster_present = monster_walk; // 播放怪物 Walk 动画
}
```

通过上述代码可实现游戏中的怪物在区域内待机和随机方向走路的 AI 功能。

3.4 状态机和行为树

游戏中玩家控制自己的角色可以用状态机来实现, 而怪物则可以用状态机和行为树来控制其 AI 行为。除了无须移动的 NPC, 游戏中的角色动作通常由状态机来控制。状态机就是多种状态的集合, 状态之间只要满足一定条件即可切换。如怪物的状态机包含行走、奔跑、攻击 1、攻击 2、受伤、死亡倒地等状态, 行走可以切换到跳跃, 奔跑可以切换到攻击 1 和攻击 2, 这里的攻击 1 和攻击 2 并非连招, 而是独立的攻击行为, 如图 6 所示。

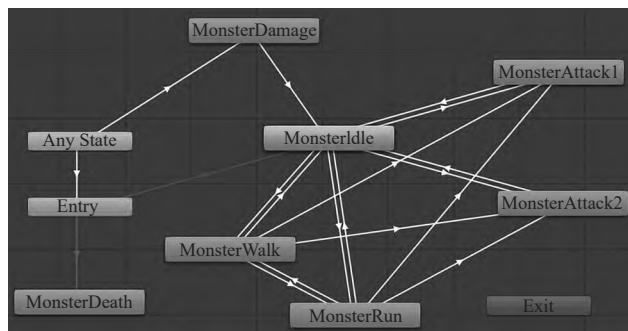


图 6 怪物状态机

由图 6 可以看出, 每个状态除了自身行为外, 还需要在适当的条件下与其他状态交互, 因此状态机形成一种网状结构。但若要实现较复杂的 AI 行为(如怪物的巡逻、围攻玩家等), 仅使用状态机是无法实现的, 这种情况需要使用 AI 行为树。行为树是树状分支节点的结构, 与状态机在结构上有较大差别。

怪物或 NPC 角色在通常情况不会发生战斗, 只有在触发适当条件时才会进入战斗状态。而所有的行为都是在行为树的范围内实现, 以此来实现 AI 控制。行为树包含控制节点和行为节点, 将所涉及的行为写成节点, 再把这些节点连成树。每次决定选择哪一种行为时, 会从根节点的各个分支节点去判定是否进入下一层节点, 找出与当前状态相符的行为。战斗

型 NPC 在没有敌人来袭时巡逻，在视野范围内出现普通敌人时则发起攻击，遇到精英怪物时则逃跑，该行为树会即时判定符合条件的条件，从而做出相应的行为，其逻辑关系与 AI 行为模块相同^[4]。这是一个简单的行为树，如图 7 所示。

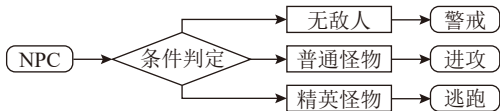


图 7 简单的行为树

行为树有多个节点满足条件时，则会按权重做出选择。如同时遇到普通怪物和精英怪物，会选择权重最高的节点。而权重以角色存活为优先，越容易令角色存活的节点权重越高^[5]，在满足某些条件的前提下，容易导致角色死亡的节点不会被选择执行。

图 5 的例子是行为树的行为节点，除此之外还有控制节点，具有以下三种类型。

3.4.1 单一选择

对于多个下层节点只选择其中一个，由于不能同时执行 2 个以上节点，最终只能选择 1 个末端节点执行。

3.4.2 顺序选择

按顺序依次选择节点并运行，其中一个节点运行完毕后再运行下一个节点。

3.4.3 同步选择

同步选择符合条件的节点并运行。如 NPC 在巡逻时与其他 NPC 交谈且进行某些肢体动作，即巡逻、交谈、肢体动作三个行为节点同时运行。前提是无敌人，敌人一旦进入视野区则会结束这三个节点的运行，进入另一个控制节点。

三种节点控制类型使用哪一种要看具体情况，不同的情景有不同的选择，有时并非单一类的运行，也有可能是多个类型混合控制。比如单一选择类型下面的分支使用了同步选择。

4 战斗系统模块的整体封装

战斗系统有诸多模块，为了便于管理和减少程序漏洞，需要对各模块实行整体封装。封装的主要思路是将各战斗模块归集到基类中。基类包含所有组件，具备战斗对象、基础字段、函数初始化的赋值等。

4.1 战斗实时信息传递

通常，玩家或怪物发出的攻击伤害和判定都会被封装在类里，在触发战斗行为事件后将信息传达给受击者。在 Unity 里可以先通过攻击碰撞触发事件，在事件发生后处理相应信息，并非是直接将战斗信息传达给受击者，这样可方便后期调试受击信息，更好地进行全局信息控制，从而减少程序漏洞的产生。

4.2 战斗系统组件的类

战斗系统以组件模式处理战斗行为，通过基类的回调函数传递战斗信息。只需将全部战斗组件的函数与 BattleCharacterObject 的回调函数关联就可以实行统一调用。以组件模式创建的战斗系统具有高度可扩展性，且能灵活定义各组件^[6]。图 8 列出了几个重要的基类，并通过基类令所有组件泛化，使组件适用于不同类型的数据。

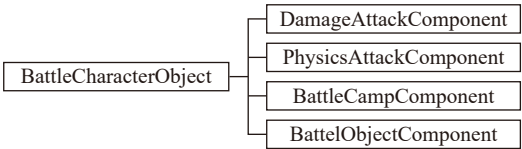


图 8 战斗系统组件的基类

其中，DamageAttackComponent（伤害攻击）和 PhysicsAttackComponent（物理碰撞）是两个重要的基类，它们处理即时碰撞后产生的事件数据。而 BattleCampComponent（战斗阵营）是一个静态类，它在战斗发生前判别对方是否为敌人，如果是敌人则可以发起战斗。其关键代码如下：

```
public static class BattleCampComponent {
    public const int PLAYER = 1;// 判别玩家
    public const int MONSTER = 2;// 判别敌方怪物
    public const int NPC = 3;// 地图上的 NPC，
    在某些条件触发下也可以转换阵营
}
```

5 结 论

对 3D 角色动作游戏的战斗系统框架、运行机制、战斗行为模块进行了研究与开发，重点研究并优化了玩家操作、AI 行为、战斗事件等内容，提升了操控流畅度。充分利用类的封装方法，将各模块整合成为一个战斗系统，最大程度简化了编程量，减少了冗余程序，提高了执行效率。

参考文献：

[1] 章国雁. 基于 Unity3D 的多人在线游戏案例设计与实现 [J]. 安徽水利水电职业技术学院学报, 2021, 21 (3) : 46-49.
[2] 张华振. 基于 Unity3D 技术的塔防游戏虚拟人物动作控制方法 [J]. 智能计算机与应用, 2022, 12 (8) : 187-189+195.
[3] 史宝明, 贺元香, 李岚. Unity3D 随机寻路算法设计 [J]. 吉林师范大学学报: 自然科学版, 2022, 43 (1) : 128-133.
[4] 张华振. 基于 Unity3D 技术的塔防游戏虚拟人物动作控制方法 [J]. 智能计算机与应用, 2022, 12 (8) : 187-189.
[5] 刘梦颖, 马宏琳. 基于 Unity 的冒险游戏设计与实现 [J]. 河南科技, 2021, 40 (17) : 30-32.
[6] 王俊. Unity 引擎下多人在线网络游戏设计及开发 [J]. 电子技术与软件工程, 2022, 230 (12) : 38-41.

作者简介：何柳青（1976.12—），男，汉族，广西北海人，讲师，硕士，研究方向：虚拟现实、游戏开发。